

Stat 4194 Project 2

Chenchenyi Zhu.3085

2025-03-28

Project 2: Supervised and Unsupervised Learning

Data Selection

Name: NCAA Basketball March Madness 2025

Data sets:

```
list.files("~/Python/Stats/STAT 4194/ncaam2025")
```

```
## [1] "coaches.csv"           "conf_team_mapping.csv" "kenpom_defense.csv"
## [4] "kenpom_efficiency.csv" "kenpom_height.csv"     "kenpom_misc.csv"
## [7] "kenpom_offense.csv"    "kenpom_pointdist.csv"  "kenpom_summary.csv"
## [10] "mm2002_2025.csv"       "postseason.csv"
```

From the provided folder of data sets, I have looked over each data set and its possible predictors and have found that some data sets, such as “kenpom_[name]” are also in the “mm2002_2025” data set. Below is a summary of which data sets each section are from:

kenpom_efficiency: Adjusted.Tempo-Avg.Possession.Length..Defense..Rank [3-18]

kenpom_defense & kenpom_offense: eFGPct-RankFTRate [19-26]

kenpom_pointdist: OffFT-RankDef3PtFG [27-38]

kenpom_summary: Tempo-RankAdjEM [39-52]

kenpom_misc: FG2Pct-RankOppNSTRate [53-85]

kenpom_height: AvgHeight-RankPGDR [86-133]

Net.Rating and Net.Rating.Rank [134-135]

conf_team_mapping: Mapped.Conference.Name-Mapped.ESPN.Team.Name [136-137]

coaches: Current.Coach-Since [138-140]

Active.Coaching.Length and Active.Coaching.Length.Index [141-142]

postseason: Seed-Final.Four. [143-150]

Top.12.in.AP.Top.25.During.Week.6. [151]

Since I want to predict the picks for the brackets, I want to narrow down the mm2002_2025.csv data set to only the year 2025 and whichever teams that have made to the post season, (can be either in the March Madness Season Tournament or be in the sorting index of 1 – this gives 68 teams for the post-tournament).

```
mm2002_2025 <- read.csv("ncaam2025/mm2002_2025.csv")

#there should be 68 teams for the final matches
keep <- mm2002_2025$Season==2025 & (mm2002_2025$Post.Season.Tournament=="March Madness" |
                                     mm2002_2025$Post.Season.Tournament.Sorting.Index==1)
ncaa25 <- mm2002_2025[keep,] #keep the variables mentioned in "keep"
dim(ncaa25)

## [1] 68 151

rownames(ncaa25) <- ncaa25$Mapped.ESPN.Team.Name
ncaa25$Seed <- as.numeric(ncaa25$Seed) #change Seed from character to numeric
```

Exploratory Analysis

Preliminary Variable Screening and Exploratory Plots:

Below we will consider several aspects of a successful college basketball team, based on the `Net.Ratings`. Those aspects include such as offense, defense, experience, coaching, level of competition, etc. We will also use adjusted variables instead of the raw variables because this way these variables take into account context, such as other variables that might influence the raw variables themselves. We will also just look at the actual values instead of the rankings, since ranking simplifies comparisons and teams ranked next to each other might differ significantly to actual performance.

Looking through the data, there are some predictors, where the values are more precise than the other ones, so I have not used `kenpom_efficiency` data, since other data set numbers are more accurate, and are not rounded, such as adjusted offensive and defensive efficiency, and adjusted tempo.

Below are some exploratory plots:

```
par(mfrow=c(3,3), cex=.66)
hist(ncaa25$Net.Rating, xlab="Net Rating", main="Histogram of Net Rating")
plot(Net.Rating~Raw.Offensive.Efficiency,
     ylab="Net Rating", xlab="Raw Offensive Efficiency", data=ncaa25)
abline(lm(ncaa25$Net.Rating ~ ncaa25$Raw.Offensive.Efficiency), col="red", lwd=2)

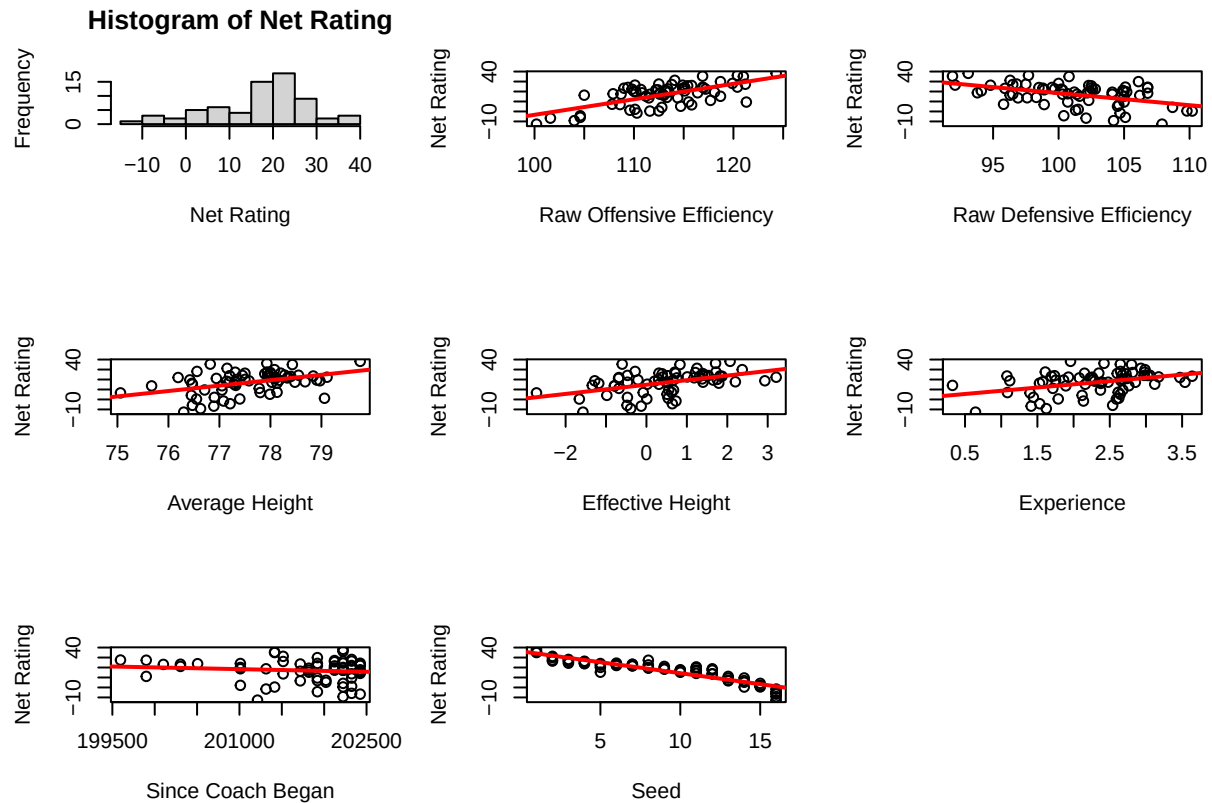
plot(Net.Rating~Raw.Defensive.Efficiency,
     ylab="Net Rating", xlab="Raw Defensive Efficiency", data=ncaa25)
abline(lm(ncaa25$Net.Rating ~ ncaa25$Raw.Defensive.Efficiency), col="red", lwd=2)

plot(Net.Rating~AvgHeight,
     ylab="Net Rating", xlab="Average Height", data=ncaa25)
abline(lm(ncaa25$Net.Rating ~ ncaa25$AvgHeight), col="red", lwd=2)

plot(Net.Rating~EffectiveHeight,
     ylab="Net Rating", xlab="Effective Height", data=ncaa25)
abline(lm(ncaa25$Net.Rating ~ ncaa25$EffectiveHeight), col="red", lwd=2)
plot(Net.Rating~Experience,
     ylab="Net Rating", xlab="Experience", data=ncaa25)
abline(lm(ncaa25$Net.Rating ~ ncaa25$Experience), col="red", lwd=2)

plot(Net.Rating~Since,
     ylab="Net Rating", xlab="Since Coach Began", data=ncaa25)
```

```
abline(lm(ncaa25$Net.Rating ~ ncaa25$Since), col="red", lwd=2)
plot(Net.Rating~Seed,
     ylab="Net Rating", xlab="Seed", data=ncaa25)
abline(lm(ncaa25$Net.Rating ~ ncaa25$Seed), col="red", lwd=2)
```

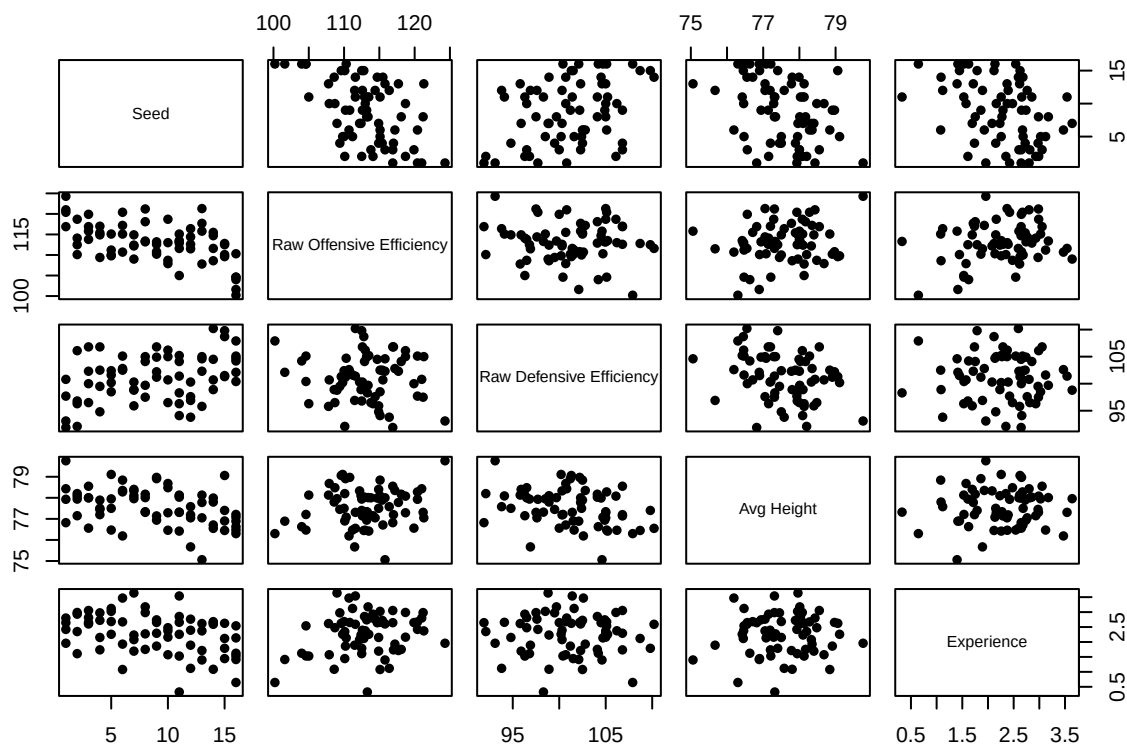


Histogram of `Net.Rating` shows a normal distribution, with most teams clustered around a moderate range of ratings.

The explanatory analysis confirms that `Raw.Offensive.Efficiency`, `Raw.Densive.Efficiency` and factors such as `Experience` and `AvgHeight` play a significant role in determining success. The `Seed` also tells about the performance of each team.

The data set includes variables related to the performance, such as `Seed`, `Raw.Offensive.Efficiency`, `Raw.Densive.Efficiency`, `Experience`, `AvgHeight` and `Net.Rating`. These variables capture critical aspects of team strength, strategy, and player composition, providing valuable feature for predictive modeling.

```
pairs(data.frame(ncaa25$Seed, ncaa25$Raw.Offensive.Efficiency,
                 ncaa25$Raw.Defensive.Efficiency, ncaa25$AvgHeight,
                 ncaa25$Experience),
      labels=c("Seed", "Raw Offensive Efficiency", "Raw Defensive Efficiency",
              "Avg Height", "Experience"), pch=16)
```



```
cor(ncaa25[, c("Seed", "Raw.Offensive.Efficiency", "Raw.Defensive.Efficiency",
               "AvgHeight", "Experience")])
```

```
##              Seed Raw.Offensive.Efficiency
## Seed          1.0000000 -0.52116734
## Raw.Offensive.Efficiency -0.5211673  1.0000000
## Raw.Defensive.Efficiency  0.3619765 -0.07767967
## AvgHeight          -0.3796622  0.18529128
## Experience          -0.3677963  0.23490867
##              Raw.Defensive.Efficiency AvgHeight Experience
## Seed              0.36197652 -0.37966218 -0.36779633
## Raw.Offensive.Efficiency -0.07767967  0.18529128  0.23490867
## Raw.Defensive.Efficiency  1.00000000 -0.31181311 -0.01941432
## AvgHeight            -0.31181311  1.00000000  0.05270084
## Experience           -0.01941432  0.05270084  1.00000000
```

Based on the exploratory plots shown above, the dependent variable with the strongest correlation to `Net.Rating` are the following: raw offensive and defensive efficiency, average height, experience, and Seed has some relationship. As such, we will subset to these variables.

Below is a small peak view of the subsetting data set for `ncaam2025/mm2002_2025.csv`:

```
cols <- which(names(ncaa25)%in%c("Seed", "Raw.Offensive.Efficiency",
                                   "Raw.Defensive.Efficiency", "AvgHeight",
                                   "Experience", "Net.Rating"))
```

```
ncaa25_subset <- ncaa25[, cols]
ncaa25_subset <- scale(ncaa25_subset, center=TRUE)
head(ncaa25_subset, 3)
```

```
##           Raw.Offensive.Efficiency Raw.Defensive.Efficiency AvgHeight Experience
## Duke                2.3606409                -1.8844208    2.4848875 -0.4761376
## Florida              1.5398175                -0.8093764    0.4066605  0.2123278
## Houston              0.8031812                -2.1648672   -0.8539035  0.5468485
##           Net.Rating      Seed
## Duke          1.885118 -1.658834
## Florida        1.707523 -1.658834
## Houston        1.636485 -1.658834
```

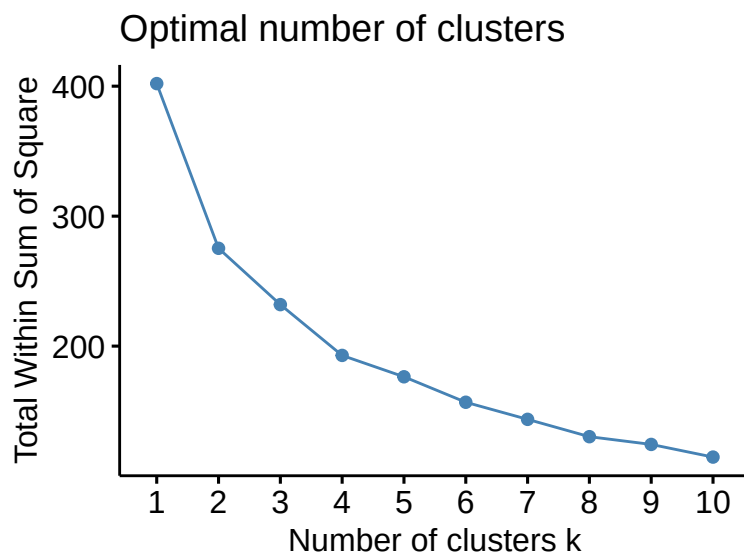
Unsupervised Learning

K-means, DBSCAN, t-SNE, Hierarchical Clustering, Principal Component Analysis (PCA)

Unsupervised learning method 1 - K-means

K-means effectively groups teams based on performance metrics, with distinct clusters highlighting differences in Net.Ratings, efficiency, and Seeds. High-performing teams naturally form clusters apart from low-performing ones.

```
#this suggest how many clusters based on 10 as most optimal
par(mar=c(2,2,2,1))
fviz_nbclust(ncaa25_subset, kmeans, method = "wss", k.max=10)
```

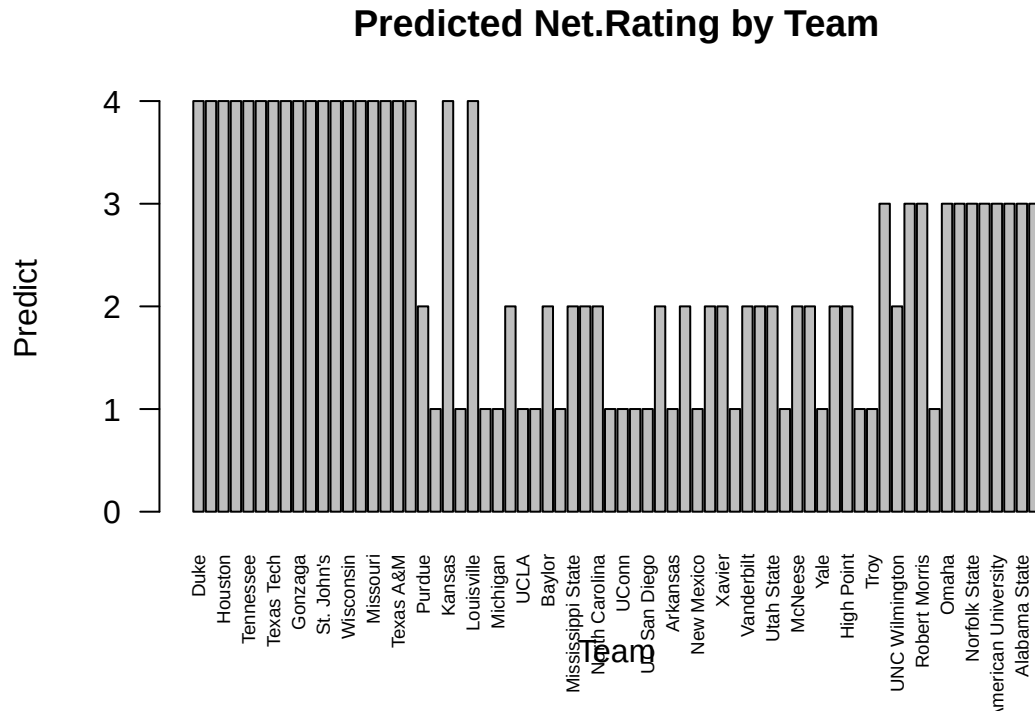


Based on the above plot, the number of clusters that would best fit the relationship between each team would be about 4 clusters.

```
set.seed(4194) #the clusters of each team will be different
out <- kmeans(ncaa25_subset, centers=4, nstart=10) #center is how many clusters want
table(out$cluster)
```

```
##
## 1 2 3 4
## 19 18 11 20
```

```
#out$cluster #which cluster belongs to
barplot(out$cluster, names.arg = rownames(ncaa25_subset),
        las = 2, main = "Predicted Net.Rating by Team",
        ylab = "Predict", xlab = "Team", cex.names = 0.6)
```



The table shows how many teams each cluster has, and it shows which team is in which cluster.

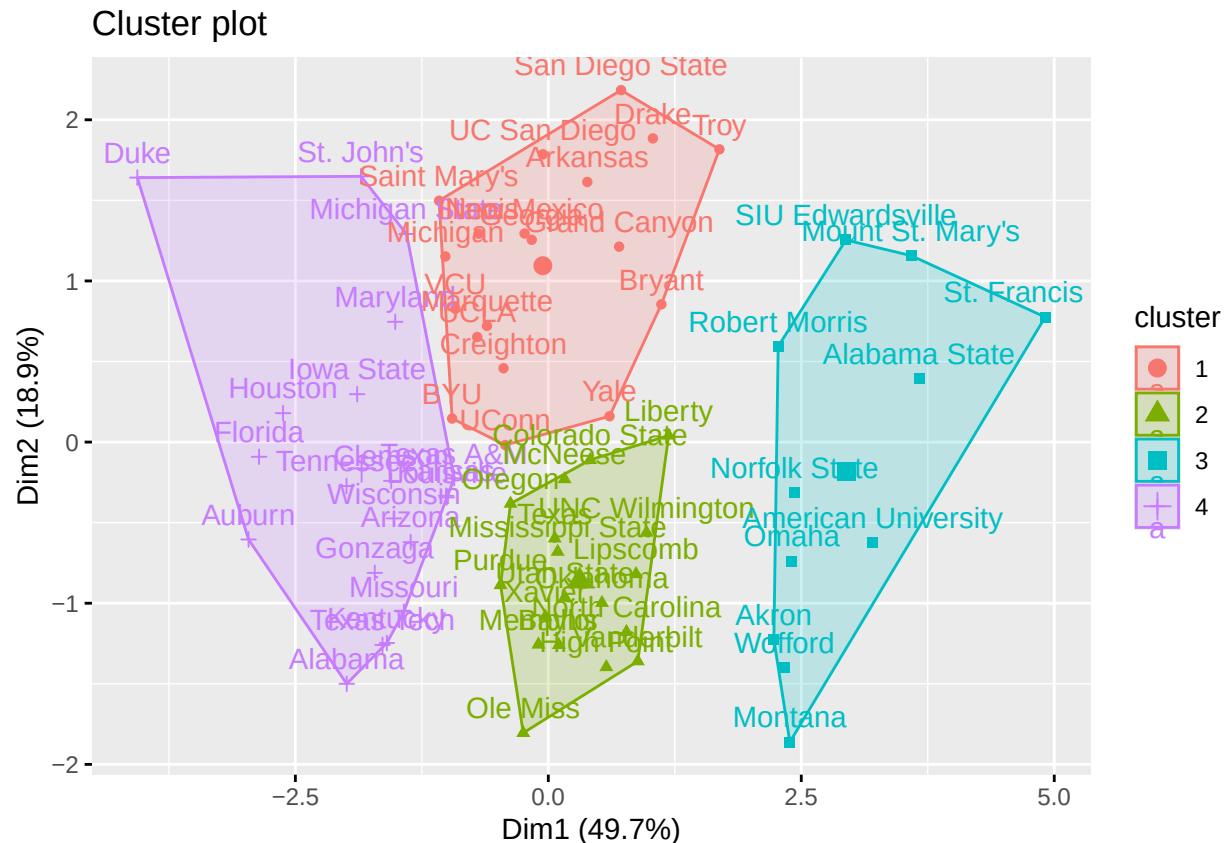
```
set.seed(4194)
out$centers
```

##	Raw.Offensive.Efficiency	Raw.Defensive.Efficiency	AvgHeight	Experience
## 1	-0.1970934	-0.4428281	0.8585269	-0.6279089
## 2	0.1413778	0.4682125	-0.6526423	0.4332932
## 3	-1.0680664	0.9859052	-1.0686433	-0.7878840
## 4	0.6474352	-0.5429524	0.3595313	0.6398857

##	Net.Rating	Seed
## 1	0.003079045	0.1894399
## 2	-0.053133374	0.2093539
## 3	-1.722480506	1.3788526
## 4	0.992259222	-1.1267554

This table shows the coordinates of where which predictor variable of each cluster is in the X and Y axis.

```
set.seed(4194)
fviz_cluster(out, data = ncaa25_subset)
```



The visual above helps to see which teams are similar based on the chosen metrics from the subsetted data. Based on the chosen k from the plot before this, we can see 4 different clusters to differentiate and cluster the teams together. We can see from the table that there are 19 teams in the first cluster, 18 teams in the second cluster, 11 teams in third cluster, and 20 teams in the fourth cluster.

It can be seen that in cluster 4, the teams have lower **Seed** value, and have higher **Raw.Offensive.Efficiency**, lower **Raw.Defensive.Efficiency**, and more **Experience**. Those teams include Duke, Houston, Auburn. However for teams in cluster 3, those teams have a higher **Seed** and **Raw.Defensive.Efficiency**, less **Experience**, and have a lower **Raw.Offensive.Efficiency**. Those teams include Montana, Omaha, Alabama State.

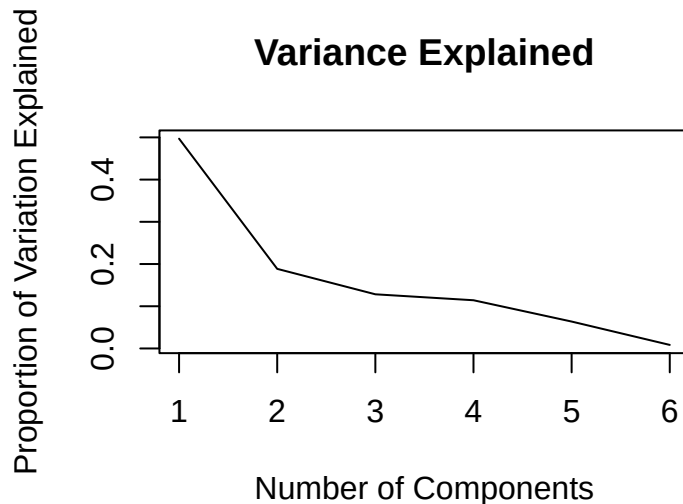
Unsupervised learning method 2 - Principal Components Analysis (PCA)

```
set.seed(4194)
pca <- prcomp(ncaa25_subset, center=TRUE, scale.=TRUE) #principal component
summary(pca)
```

```
## Importance of components:
##              PC1    PC2    PC3    PC4    PC5    PC6
## Standard deviation  1.7267 1.0636 0.8772 0.8277 0.61819 0.2246
```

```
## Proportion of Variance 0.4969 0.1885 0.1283 0.1142 0.06369 0.0084
## Cumulative Proportion 0.4969 0.6855 0.8137 0.9279 0.99160 1.0000
```

```
plot(1:6, summary(pca)$importance[2,], xlab="Number of Components", cex=0.6,
     ylab="Proportion of Variation Explained", main = "Variance Explained", type="l")
```



In the table above, the Proportion of Variance explains the variance each principal component as a percentage of total variance. PC1 explains 68.86% of the variance, while PC2 explains 13.86% of the total variance, and it all decreases afterwards, shown in the plot below.

Here, it can be seen that it shows the number of principal components to use. It seems best to choose 5 since 6 and 7 seem to not have much difference in the cluster and are at 0 difference already. 2 cluster might also be reasonable here, where there is a large variation.

```
set.seed(4194)
n <- nrow(ncaa25_subset)
cols <- sample(c("red", "blue", "green", "black"),
               size=n, replace=TRUE)

c1 <- c(mean(pca$x[cols=="red",1]), mean(pca$x[cols=="red",2]))
c2 <- c(mean(pca$x[cols=="blue",1]), mean(pca$x[cols=="blue",2]))
c3 <- c(mean(pca$x[cols=="green",1]), mean(pca$x[cols=="green",2]))
c4 <- c(mean(pca$x[cols=="black",1]), mean(pca$x[cols=="black",2]))

# function that finds distance from each point
# to a center
f1 <- function(pcac, r){
  return(sqrt(sum((r-pcac)^2)))
}

for(i in 1:11){
  ds <- cbind(apply(pca$x, 1, f1, r=c1),
              apply(pca$x, 1, f1, r=c2),
              apply(pca$x, 1, f1, r=c3),
              apply(pca$x, 1, f1, r=c4))
  labs <- apply(ds, 1, which.min)
```



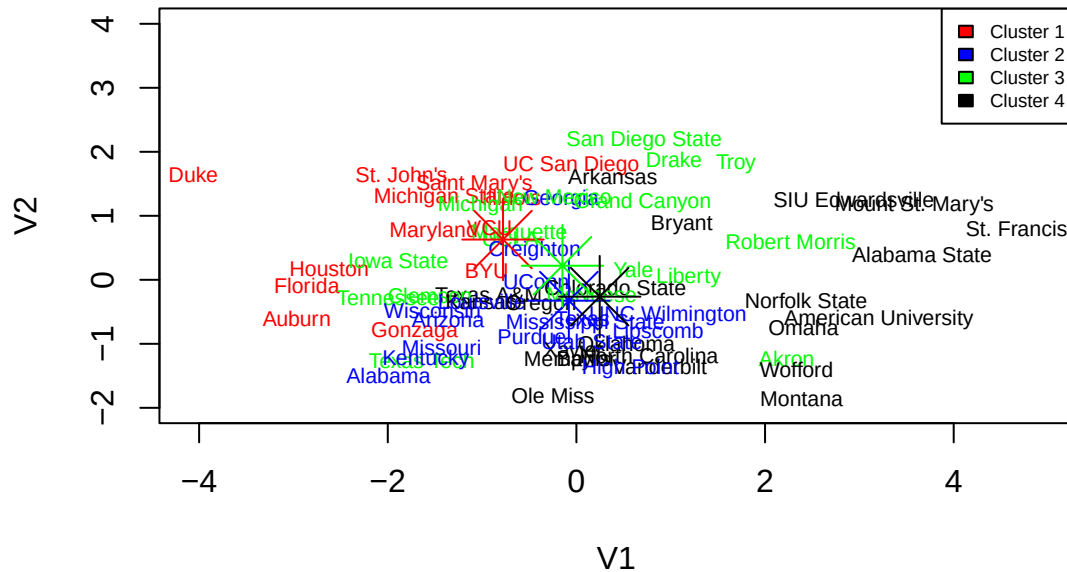
```

cols <- replace(replace(replace(replace(labs, labs==1, "red"), labs==2, "blue"),
                                labs==3, "green"), labs==4, "black") # Include black for c4
if (any(cols == "red")) {
  c1 <- apply(ncaa25_subset[cols == "red", ], 2, mean)
} else {
  c1 <- NA # Handle empty cluster
}
if (any(cols == "blue")) {
  c1 <- apply(ncaa25_subset[cols == "blue", ], 2, mean)
} else {
  c1 <- NA # Handle empty cluster
}
if (any(cols == "green")) {
  c1 <- apply(ncaa25_subset[cols == "green", ], 2, mean)
} else {
  c1 <- NA # Handle empty cluster
}
if (any(cols == "black")) {
  c1 <- apply(ncaa25_subset[cols == "black", ], 2, mean)
} else {
  c1 <- NA # Handle empty cluster
}
}

plot(pca$x[,1], pca$x[,2], xlab="V1", ylab="V2",
     main=paste0("Step ",i, ": Move Points to Nearest\nCentroid's Cluster"),
     col="white", ylim=c(-2,4))
text(pca$x[,1], pca$x[,2], rownames(ncaa25_subset), cex=.66, col=cols)
points(c1[1], c1[2], col="red", pch=8, cex=4)
points(c2[1], c2[2], col="blue", pch=8, cex=4)
points(c3[1], c3[2], col="green", pch=8, cex=4)
points(c4[1], c4[2], col="black", pch=8, cex=4)
legend("topright", legend = c("Cluster 1", "Cluster 2", "Cluster 3", "Cluster 4"),
      fill=c("red", "blue", "green", "black"), cex=0.6)

```

Step 11: Move Points to Nearest Centroid's Cluster



After scaling the data, I have computed the eigenvalues of the p-dimensional matrix, with the largest eigenvalue explaining the column space that contains the most information, and removing the least informative.

I have used 4 clusters for the plotted data because the variance explained seems to best fit this data. From the plot it can be seen that teams that performed better are in cluster 1, and teams that performed worst are in cluster 4.

Supervised Learning

Non-linear Regression, Splines Regression, Principal Components Regression, LASSO/Ridge, Random Forest, Classification/Regression Trees, XGBoost, Naive Bayes, K-nearest Neighbors

I will take Net.Rating as the dependent variable in the supervised learning.

I will use the same subsetted data set for `ncaa2025/mm2002_2025.csv`, which is called `ncaa25_subset`.

Below is the training and testing sets:

```
set.seed(4194)

n <- nrow(ncaa25) #num of rows in dataset (original)

#generate training/testing set
trainrows <- sample(1:n, size=trunc(.7*n), replace = FALSE)
testrows <- (1:n)[-trainrows]

#select variables of interest
cols <- which(names(ncaa25)%in%c("Seed", "Raw.Offensive.Efficiency",
                                "Raw.Defensive.Efficiency", "AvgHeight",
                                "Experience", "Net.Rating", "Mapped.ESPN.Team.Name"))
```

```

train <- ncaa25[trainrows, cols]
test <- ncaa25[testrows, cols]

ntrain <- nrow(train)

dummy_var_train <- dummyVars(~Mapped.ESPN.Team.Name, data=train, fullRank=TRUE)
ohe_train <- predict(dummy_var_train, train)
dummy_var_test <- dummyVars(~Mapped.ESPN.Team.Name, data=test, fullRank=TRUE)
ohe_test <- predict(dummy_var_test, test)

#one-hot encoded feature
train <- cbind(train, ohe_train)
test <- cbind(test, ohe_test)

```

Supervised learning method 1 - LASSO/Ridge

```

#lasso, alpha=1
out_lasso <- cv.glmnet(as.matrix(train[,c(1,2,3,4,7)]), train$Net.Rating,
                      nfolds=10, type.measure = "deviance", alpha=1)
preds_lasso <- predict(out_lasso, as.matrix(test[,c(1,2,3,4,7)]))
(RMSPE_lasso <- sqrt(mean((preds_lasso-test$Net.Rating)^2)))

```

```
## [1] 4.963823
```

```
coef(out_lasso)
```

```

## 6 x 1 sparse Matrix of class "dgCMatrix"
##                               s1
## (Intercept)                4.2520035
## Raw.Offensive.Efficiency    0.3784864
## Raw.Defensive.Efficiency   -0.1688329
## AvgHeight                   0.0191959
## Experience                   .
## Seed                       -1.6805489

```

From the LASSO output shown, Experience does not seem to have a big influence in predicting the Net.Rating, while variables such as Raw.Offensive.Efficiency and AvgHeight have a positive influence and Raw.Defensive.Efficiency and Seed have a negative influence on Net.Rating.

The root mean squared error (RMSPE) for LASSO is 4.598006, which indicates typical deviation of the model's predictions from the actual Net.Rating.

```

#ridge, alpha=0
out_ridge <- cv.glmnet(as.matrix(train[,c(1,2,3,4,7)]), train$Net.Rating,
                      nfolds=10, type.measure = "deviance", alpha=0)
preds_ridge <- predict(out_ridge, as.matrix(test[,c(1,2,3,4,7)]))
(RMSPE_ridge <- sqrt(mean((preds_ridge-test$Net.Rating)^2)))

```

```
## [1] 4.169738
```

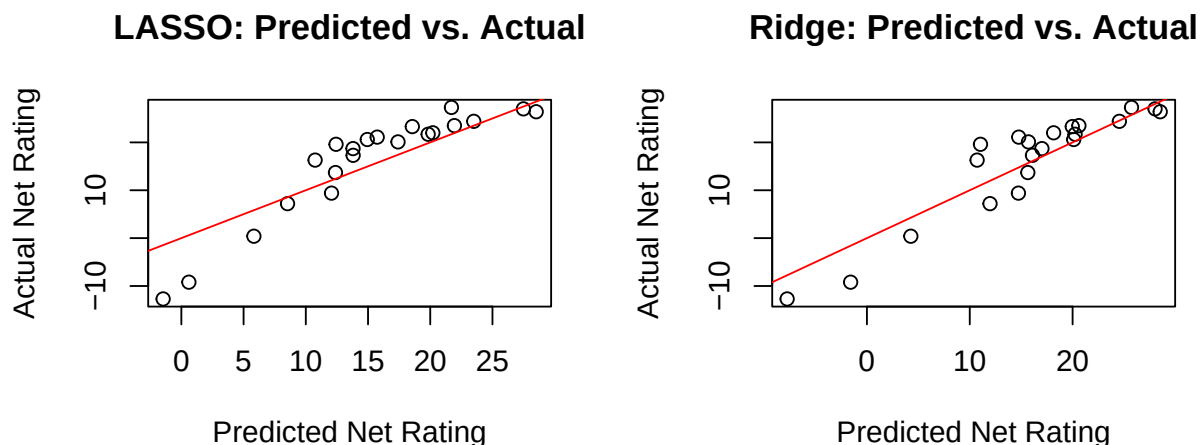
```
coef(out_ridge)
```

```
## 6 x 1 sparse Matrix of class "dgCMatrix"
##                               s1
## (Intercept)                 -89.4430732
## Raw.Offensive.Efficiency     0.6609244
## Raw.Defensive.Efficiency    -0.4952820
## AvgHeight                    1.1481344
## Experience                    1.4831643
## Seed                        -1.2286031
```

From the Ridge output shown, variables such as `Raw.Offensive.Efficiency`, `AvgHeight`, `Experience` have a positive influence and `Raw.Defensive.Efficiency` and `Seed` have a negative influence on `Net.Rating`.

The root mean squared error (RMSPE) for Ridge is 4.307337, which indicates typical deviation of the model's predictions from the actual `Net.Rating`. This is slightly better than the previous RMSE of 4.598006, suggesting improved predictive accuracy. It can be seen in the plot below that the predictions in Ridge model is much better than predictions in LASSO.

```
par(mfrow=c(1,2))
plot(preds_lasso, test$Net.Rating,
     xlab = "Predicted Net Rating", ylab = "Actual Net Rating",
     main = "LASSO: Predicted vs. Actual")
abline(0, 1, col = "red") # diagonal line
plot(preds_ridge, test$Net.Rating,
     xlab = "Predicted Net Rating", ylab = "Actual Net Rating",
     main = "Ridge: Predicted vs. Actual")
abline(0, 1, col = "red") # diagonal line
```



In this method, it is necessary to improve `Raw.Offensive.Efficiency` and reducing `Raw.Defensive.Efficiency` to enhance team performance. While teams with better `Seed` and taller players are also more likely to perform better.

Supervised learning method 2 - XGBoost

The XGBoost method iterates through each tree and find the more important variables and re-iterate each tree. With one-hot encoding using dummy variables, team names are transformed into numeric representations for the XGBoost model.

```
#XGBoost
#exclude dependent(net rating) and team names
train_matrix <- as.matrix(train[, c(1,2,3,4,7)])
test_matrix  <- as.matrix(test[, c(1,2,3,4,7)])

dtrain <- xgb.DMatrix(data=train_matrix, label=train$Net.Rating)
dtest  <- xgb.DMatrix(data=test_matrix, label=test$Net.Rating)

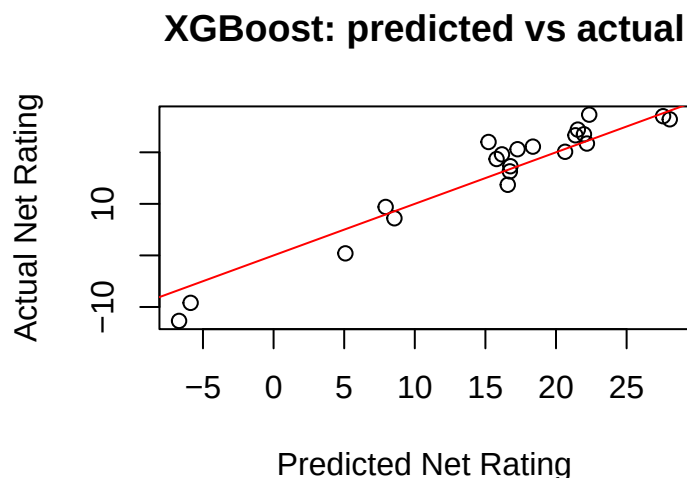
#parameter for xgboost
params_reg <- list(objective = "reg:squarederror", eval_metric="rmse")
#xgboost model
xgb_model <- xgb.train(params = params_reg, data=dtrain,
                      nrounds = 100, verbose = 1)

#predict
preds <- predict(xgb_model, newdata=dtest)
(RMSPE <- sqrt(mean((preds-test$Net.Rating)^2)))
```

```
## [1] 3.153205
```

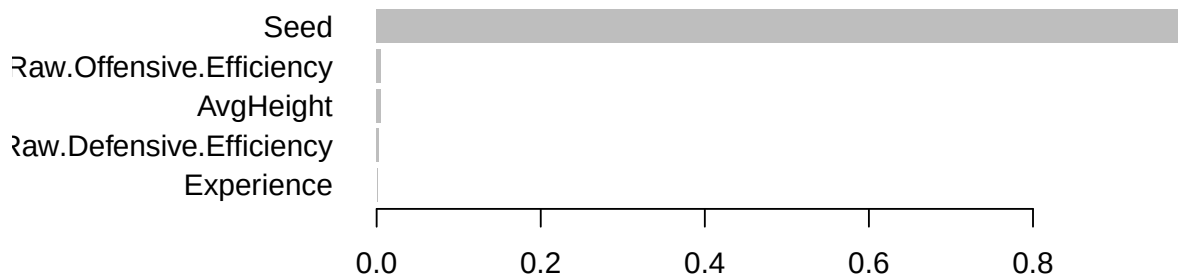
In this model, the root mean square error (RMSE) is 3.153205, which indicates that the model has reasonable predictive accuracy with a strong prediction performance.

```
#predict vs actual
plot(preds, test$Net.Rating,
     xlab="Predicted Net Rating", ylab="Actual Net Rating",
     main="XGBoost: predicted vs actual")
abline(0, 1, col="red")
```



The scatter plot of predicted vs actual `Net.Rating` also shows high accuracy for predictions, which are points that are closely to the diagonal line, where it indicates that predicted values are close or match to the actual values.

```
#shows importance of predictors
importance_matrix <- xgb.importance(feature_names = colnames(train_matrix), model = xgb_model)
xgb.plot.importance(importance_matrix)
```



From the importance bar chart, it can be seen that the `Seed` has the highest importance in predicting `Net.Rating` compared to the other variables, with `Raw.Defensive.Efficiency` and `Experience` having the minimal to no importance.

Supervised learning method 3 - Classification Tree

```
set.seed(4194)
ncaa25_subset <- scale(ncaa25_subset, center=TRUE)

f1 <- randomForest(Net.Rating ~Seed+Raw.Offensive.Efficiency+
                  Raw.Defensive.Efficiency+AvgHeight+Experience,
                  data=ncaa25_subset)
f1
```

```
##
```

```
## Call:
```

```
## randomForest(formula = Net.Rating ~ Seed + Raw.Offensive.Efficiency +
```

```
##               Type of random forest: regression
```

```
##               Number of trees: 500
```

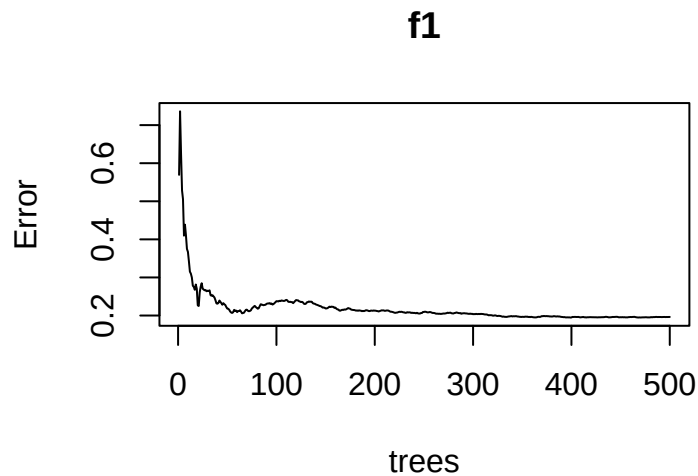
```
## No. of variables tried at each split: 1
```

```
##
```

```
##               Mean of squared residuals: 0.1962537
```

```
##               % Var explained: 80.08
```

```
plot(f1)
```

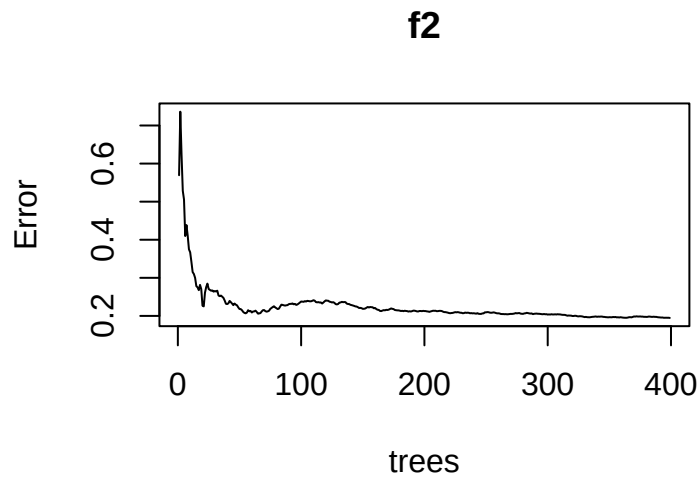


The plot also shows that as the number of trees increases the error decreases significantly. It shows that it slows down a 150-300 trees, it might indicate that there is no need to compute 500 times to find average, and that adding more trees does not reduce much of the error, so the model can be cut down to around `ntree=300-400`.

```
set.seed(4194)
f2 <- randomForest(Net.Rating ~Seed+Raw.Offensive.Efficiency+
                   Raw.Defensive.Efficiency+AvgHeight+Experience,
                   data=ncaa25_subset, ntree=399)
f2
```

```
##
## Call:
## randomForest(formula = Net.Rating ~ Seed + Raw.Offensive.Efficiency +      Raw.Defensive.Efficiency
##               Type of random forest: regression
##               Number of trees: 399
## No. of variables tried at each split: 1
##
##               Mean of squared residuals: 0.1947348
##               % Var explained: 80.24
```

```
plot(f2)
```

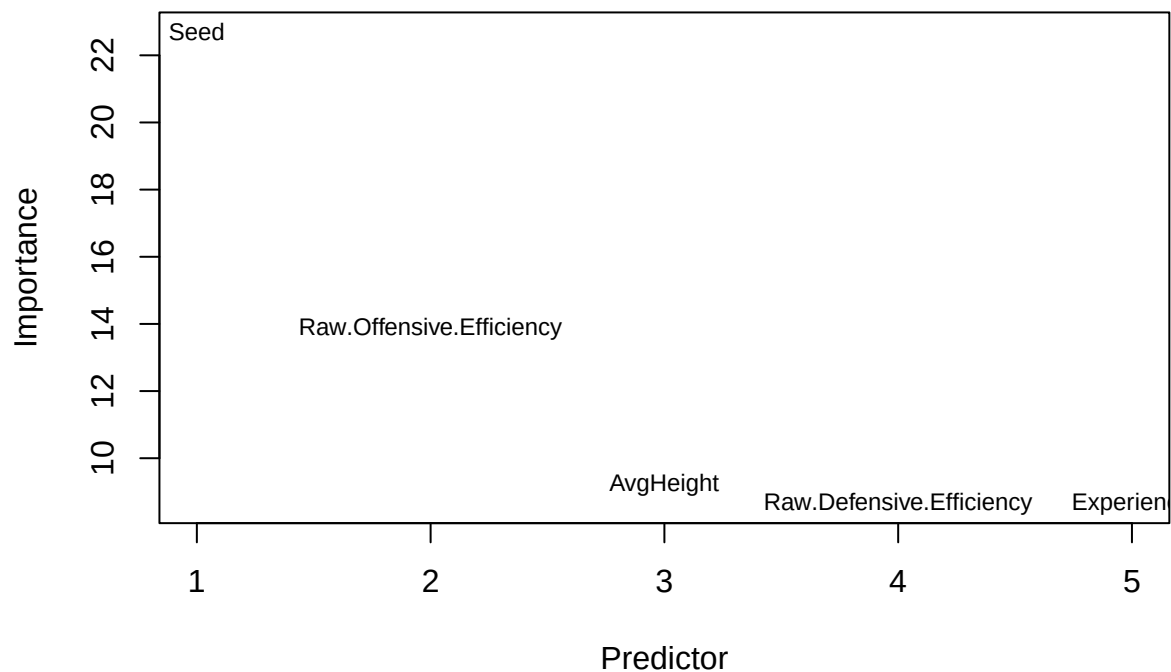


This error plot shows a better decline of error rate for 100 trees. However, the Mean of squared residuals is 0.1947734, which is better than the first model plotted, which has a Mean of squared residuals of 0.196204. The variance percentage is also higher for the first model, which might not be good at capturing the relationships. Thus I will use the second model.

```
importance(f2)
```

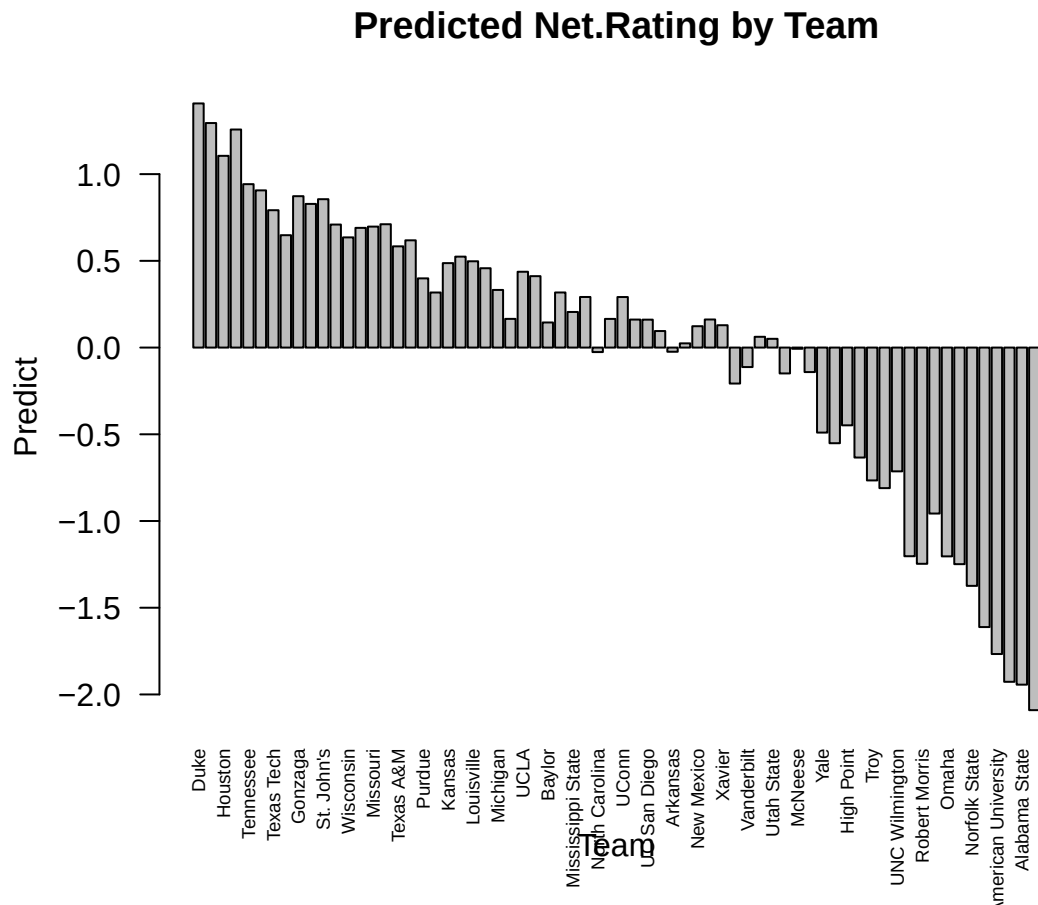
```
##                               IncNodePurity
## Seed                         22.713684
## Raw.Offensive.Efficiency     13.862359
## Raw.Defensive.Efficiency     8.630368
## AvgHeight                    9.198166
## Experience                    8.629497
```

```
plot(1:length(importance(f2)),
     importance(f2)[order(importance(f2), decreasing=TRUE)],
     ylab="Importance", xlab="Predictor", col="white")
text(1:length(importance(f2)),
     importance(f2)[order(importance(f2), decreasing=TRUE)],
     rownames(importance(f2))[order(importance(f2), decreasing=TRUE)], cex=.75)
```

From the importance function, the `Seed` variable has a much more important role in predicting the `Net.Rating`, followed by `Raw.Offensive.Efficiency` and then `Avg.Height`, `Raw.Defensive.Efficiency` and lastly `Experience` with the lowest importance.

```
p2 <- predict(f2, ncaa25_subset)
barplot(p2, names.arg = rownames(ncaa25_subset),
        las = 2, main = "Predicted Net.Rating by Team",
        ylab = "Predict", xlab = "Team", cex.names = 0.6)
```



From this bar plot it can be seen that teams that are performing well based on their **Net.Rating** are in the left side of the x-axis, showcasing strong performance. While teams such as Alabama State, Omaha, have weak performance, which are indicated by negative predictions.

Analysis and Findings

`ncaa25_subset` is the sub-setted data set of the 2025 NCAA men basketball teams. This data set has only the variables **Seed**, **Raw.Offensive.Efficiency**, **Raw.Densive.Efficiency**, **Experience**, **AvgHeight** and **Net.Rating**. In the exploratory analysis, the plots show good correlations between **Net.Rating** and the predictors listed. > Offensive and defensive efficiency showed opposing effects, with offensive efficiency having a positive correlation and the defensive efficiency having a negative correlation to **Net.Rating**.

Unsupervised Learning:

For K-means and PCA, the optimal number of clusters are both about 4, with a large variance between 1 and 2. K-means assigned teams into clusters based on performance metrics, with high-performing and low-performing teams clustered separately. It can be seen that teams such as Duke, Auburn, Houston are all clustered in the better performing teams, with a lower **Seed** value, and do better in **Raw.Offensive.Efficiency**, lower **Raw.Defensive.Efficiency**, and have more **Experience**. However for lower performing teams, such as St. Francis, Omaha, Alabama State, those teams have a higher **Seed** and **Raw.Defensive.Efficiency**, less **Experience**, and have a lower **Raw.Offensive.Efficiency**. These teams mentioned are also similar to those of PCA. Both methods are clustered similarly for most teams.

Supervised Learning:

For the LASSO and Ridge methods, the Ridge model improves RMSE slightly compared to the prior LASSO output(4.307 vs 4.598). It also includes **Experience** as a significant variable, suggesting that in this iteration, experience plays a stronger role in determining team performance.

For XGBoost, the RMSE is 3.153205, and comparing it to the previous model, Ridge and LASSO, it is doing better at predicting accuracy of the **Net.Rating**. The importance model also shows that **Seed** is a strong predictor in predicting the **Net.Rating** of different teams compared to the other variables.

For Classification Tree, the second model with 399 trees shows the best at estimating the **Net.Rating** from the other predictor variables, with a RMSE of only 0.1947348.

In conclusion, for teams to achieve a higher **Net.Rating**, they should focus on enhancing **Raw.Offensive.Efficiency**, while managing **Raw.Defensive.Efficiency**. They should practice more on shooting accuracy and optimize player rotations for defensive improvements. Teams with taller average height and more experience have a higher chance in obtaining a higher **Net.Rating**.

Compare to bracket:

Comparing to my bracket, I think I did a pretty good job at maintaining with having lower **Seed** values to move forward, but later on, it was difficult to choose which teams might move forward since their values were really similar.